

This report describes the various operations implemented according to the AE2 pdf.

Part1:

Binary Search Tree (BST) is used to represent a dynamic set ADT that stores distinct integer elements. The operations include standard set operations such as addition, removal, search, as well as set-theoretical operations like union, intersection, difference, and subset checking

Analysis of each operation

- **ADD(S,x):** adds element x to S , if it is not present already.
This operation inserts a new node with the specified key into the BST. It starts at the root and traverses down the tree until it finds the correct leaf position for the new node.
Runtimes: In the worst case when all nodes in the BST have only one child then the run time is $O(n)$ (skewed tree). However in the average case $O(h)$ (where h is the height of the tree) when assuming the BST remains balanced. In the best case it takes $O(1)$ time when the tree is empty and the node is inserted at the root.
- **REMOVE(S,x):** remove element x from S , if it is present.
This operation handles 3 cases which are
 - No left child: Replace node with its right child.
 - No right child: Replace node with its left child.
 - Two children: Replace node with its in-order successor (found using the minimum operation) and adjust pointers accordingly.Runtime: Worst case $O(n)$ when all nodes in the tree only have one child
Average case $O(\log n)$ for a balanced tree. In the best case where x is the root then its $O(1)$ time
This operation also employs helper methods Minimum and Transplant.
Minimum works by repeatedly traversing the left child of subtree of the passed in node until reaching the last node. Its run time is $O(h)$ where h is the height of the subtree. In the worst case it is $O(n)$ when the sub tree is skewed. Average case is $O(\log n)$
REMOVE also calls helper method TRANSPLANT() which replaces the subtree rooted at node 1 with the subtree rooted at node 2. Runtime is $O(1)$
- **IS-ELEMENT(S,x):** check whether element x is in set S . Runtime $O(h)$, which is $O(n)$ in the worst-case (skewed tree) and $O(h)$ on average for a balanced tree. Best case its $O(1)$ if x is the root
- **SET-EMPTY(S):** check whether set S has no elements. Runtime is $O(1)$
- **SET-SIZE(S):** return the number of elements of set S . Accomplished by recursively counting the number of nodes in the tree. Runtime is $O(n)$
- **UNION(S,T):** return the union of sets S and T . Creates a new BST which contains every unique element present in either Tree S or T . It calls the helper method unionInsert twice. Firstly to add all of the unique elements from S into the new tree and then to add all the unique elements of T into the new tree. Then returns the newly made BST.

unionInsert is a recursive function which performs an inorder traversal on the BST and inserts every encountered node into the target BST. Makes use of the ADD method to only add an element if it doesn't already exist in the new BST

Runtime depends on the size of the 2 input trees. On average traversal is $O(\log n)$ when both trees are balanced so the average case is approximately $O((n_1 + n_2) \log (n_1 + n_2))$. In the worst case with skewed trees is $O(n)$ resulting in a worst-case complexity of $O((n_1 + n_2)^2)$

- **INTERSECTION (S,T):** return the intersection of sets S and T
The INTERSECTION method constructs a new BST U that contains only the elements present in both input BSTs S and T.
Creates a new BST which stores the common elements, then calls the helper method intersectionInsert. This performs an in order traversal of S and checks if the nodes key exists in T using IS_ELEMENT. If the key is found then it is added to U
intersectionInsert visits each node of the tree. Firstly recursively traverses the left subtree and checks membership in tree T, then repeats on the right tree.

Traversing tree S takes $O(n_1)$ time. For each node, IS_ELEMENT is called on tree T. In the worst case, if T is unbalanced, each lookup takes $O(n_2)$, giving a total of $O(n_1 \times n_2)$. If T is balanced, lookups take $O(\log n_2)$ on average, resulting in $O(n_1 \times \log n_2)$. Inserting into the result tree U is $O(\log m)$ per insert, but occurs only for matching nodes and doesn't affect the overall complexity. Worst case: $O(n_1 \times n_2)$ and average case $O(n_1 \times \log n_2)$

- **DIFFERENCE(S,T):** returns the difference of sets S and T
DIFFERENCE method constructs a new BST that contains only the elements present in BST S that are not found in BST T.
Works by firstly creating a new BST U to hold resulting elements. Then differenceInsert is called to perform an inorder traversal of S. For each node in S, it checks if the key is absent in T (using the IS_ELEMENT method). If the key is not found in T, it is inserted into U.

differenceInsert recursively traverses the left subtree, processes the current node, and then traverses the right subtree. This ensures that each element of S is visited in ascending order. For each node, the method uses T.IS_ELEMENT(T, node.key) to determine if the key exists in T. If the key is not present in T, it inserts the key into U using the BST's ADD method.

Runtime: Traversal of S takes $O(n_1)$ time. The membership test takes $O(\log n_2)$ on average and $O(n_2)$ in the worst case.

Insertion in U takes $O(\log m)$. Overall worst case is $O(n_1 * n_2)$ and average case is $O(n_1 * \log n_2)$

- **SUBSET(S,T):** check whether set S is a subset of set T
Achieved using a helper method that recursively traverses S and verifies the membership of each node's key in T.
SUBSET calls subsetHelper with the root of S and the tree T. For each non-null node, it checks if the node's key exists in T using T.IS_ELEMENT(T, node.key). If any key is not found, the function immediately returns false. Then recursively checks the left and right subtrees of the current node. The node is considered part of a subset only if both subtrees also satisfy the subset condition.

RunTime, Traversal of S takes $O(n_1)$ time. Membership test in T takes on average $O(\log n_2)$ and worst case $O(n_2)$. So overall the complexity is Average-case: $O(n_1 * \log n_2)$ and worst case $O(n_1 * n_2)$

- b) The naive union implementation results in an overall runtime of $O(n_1 \log n_2)$ (assuming T is balanced) or worse if T is skewed. A better implementation leverages the fact that an in-order traversal of a BST produces a sorted sequence. This takes 3 steps, firstly in order traversal of both BST S and T to extract their elements into two sorted arrays or lists. This takes $O(n_1)$ time for S and $O(n_2)$ time for T. Secondly, Merge the two sorted sequences into one sorted list while removing duplicates. can be performed in $O(n_1 + n_2)$ time. Thirdly build a balanced BST from the merged sorted list. Done by recursively selecting median as the root, proceeding to do the same for the left and right subarrays. takes $O(n_1 + n_2)$ time.
Overall complexity runs in $O(n_1 + n_2)$

Instructions to run my code

For part1 to run the code you do as follows:

You create a variable of type BSTree by writing: `BSTree S = new BSTree();`

Then you can perform different operations mentioned above, for example to add a element to the Tree you write `S.add(S, INTEGER);`

`INORDER(BSTree.root);` is the method used to print out all the elements in the tree.

To perform operations on 2 trees like union, intersect etc... then you call one of the set operations with both trees as parameters. For example: `BSTree union = BSTree.UNION(S, T);`

For part2 to run the code you do the following:

Firstly declare an integer array of rope lengths, then create a minimum priority queue of size equal to the number of ropes: `MinPriorityQueue mpq = new MinPriorityQueue(INTEGER);`. Then You can create a loop which iterates over the elements in the array and adds them to mpq. Done by `mpq.insert(INTEGER);`. Now to get the lowest cost to connect all the ropes you create another loop which iterates until mpq is empty. Every iteration remove the two smallest rope lengths by calling `mpq.extractMin()` on two variables. Compute the sum of these numbers, save it to a running total and insert it into the min priority queue. `Mpq.insert(cost);`

Part2:

My implementation solves the rope connection problem by using a Min-Priority Queue implemented as a binary heap with a dynamic resizable array. The min priority queue supports efficient insertion, extraction of the smallest element and peeking at the smallest element.

Works by inserting all rope lengths into the priority queue. My algorithm repeatedly extracts two smallest rope lengths and combines them. The new cost of combining the ropes is inserted into the priority queue again. Process continues until only one rope remains, accumulated cost from each connection step gives the total minimum cost for connecting all ropes.

The min priority queue is needed because it allows efficient retrieval and removal of the minimum element in $O(\log n)$ time. Otherwise we would be required to traverse the whole dataset and extract smallest element taking $O(n)$ time. This would need a operation per extraction so result in $O(n^3)$.

The timecomplexity for my implementation is $O(n \log n)$ to build the min heap, $O(n \log n)$ to merge the ropes. Thus the overrall time complexity is $O(n \log n)$.

- c) output for this instance of the problem 4,8,3,1,6,9,12,7,2 is 153